

# Framework / screen boundary

---

Companion to [modular-monolith.md](#). That doc covers the **vertical** split of the business layer into per-domain modules; this one covers the **horizontal** split between the reusable web framework and the screens built on it.

**Status:** Done and compiler-enforced. `Tf1Cbs.Framework` is its own assembly and references **only** `Tf1Cbs.Abstractions` (+ `Tf10mniLog` / `Tf10mniDb`) — it can reach neither a screen nor a business module. Last reviewed 2026-08-21.

## The rule, in one line

---

`Tf1Cbs.Framework` = web plumbing (HTTP / session / routing / access guard / search modal / tokens / pickers) · service modules = data & business ( `Result` -returning, zero ASP.NET) · web RCLs = screens · hosts = composition roots.

Dependencies run strictly inward: hosts → web RCLs → Web.Shared / Framework → service modules → `Tf1Cbs.Abstractions` . Nothing points back up.

## Shape (current)

---

The framework is a plain class library with `FrameworkReference Microsoft.AspNetCore.App` . App developers consume it as an opaque dependency: one `builder.Services.AddCbsFramework(config)` + one `app.UseCbsFramework()` (extension methods in `Tf1Cbs.Framework/DependencyInjection/CbsFrameworkExtensions.cs` , surfaced via the `Microsoft.Extensions.DependencyInjection` / `Microsoft.AspNetCore.Builder` namespaces so no `using` is needed). The individual services and middleware are not wired by hand and not visible to screen code.

**Four tiers, not three** — the original two-tier `Framework` / `UI` shape grew a shared-view RCL and a set of per-domain screen RCLs as the module split landed:

| Tier              | Projects                                                                                         | Holds                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------|--------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Framework</b>  | TflCbs.Framework                                                                                 | RowToken , Notify , CbsAccessAttributes , CbsAccessMiddleware , CbsSessionStore , MenuService , SearchEndpoint , SearchDescriptor , RecordPickerModel / AccountPickerModel / DatePickerModel , EditViewModel , CbsScreenController , DatabaseSelector + multi-provider DataAccess wiring, IReferenceCache , security headers, forwarded headers, antiforgery, the cbs-login rate limiter, health checks, AddCbsObservability , and SSO (CbsAuth cookie + DataProtection key ring + revocation). |
| <b>Shared web</b> | TflCbs.Web.Shared (RCL, no controllers)                                                          | Layout, _ViewImports / _ViewStart , the shared partials ( _RecordPicker , _AccountPicker , _DatePicker , _SearchModal , _Notification , _ActionBar , _DbSwitcher ) and all shared static assets, served from ~/_content/TflCbs.Web.Shared/ .                                                                                                                                                                                                                                                    |
| <b>Screens</b>    | TflCbs.Core.Authentication.Web , TflCbs.Core.Shell.Web , and five TflCbs.Modules.<Area>.Web RCLs | Controllers, views, view models and per-domain JS. Each carries an <Area>WebModuleMarker so a host can AddApplicationPart it.                                                                                                                                                                                                                                                                                                                                                                   |
| <b>Hosts</b>      | TflCbs.Host.Main + four TflCbs.Host.<Domain> + TflCbs.Gateway                                    | Program.cs , config, composition. TflCbs.Host.Main 's own Views/ / wwwroot/ are empty but for _ViewImports.cshtml .                                                                                                                                                                                                                                                                                                                                                                             |

The business tier sits below all of this and is covered by [modular-monolith.md](#) — nine service-module assemblies, none of which the framework can see.

## Namespaces

**Namespace == assembly**, no exceptions. Framework types live under TflCbs.Framework. , *screen types under TflCbs.Modules.<Area>.Web. ,* and so on, so a using names the assembly you are coupling to. The one sanctioned deviation is extension-method homes in Microsoft.Extensions.DependencyInjection / Microsoft.AspNetCore.Builder . Enforced by NamespaceDisciplineTests (which also forbids two assemblies sharing a namespace).

(Historical note: framework types were originally left in the TflCbs.Host.Main. namespace on the argument that the assembly boundary, not the namespace, does the enforcing. That was reversed — with nine module assemblies and eight RCLs, a using that lies about its assembly costs more than the rename did.)\*

## Encapsulation (enforced hiding)

- **"Framework must not depend on screens" is compiler-enforced.** Framework has no project

reference to any host or RCL; adding one would be a circular reference the compiler rejects.

- **"Framework must not depend on a business module" is also compiler-enforced.** The framework references only `Tf1Cbs.Abstractions`, `Tf1OmniLog` and bare-DLL `Tf1OmniDb`. Where it genuinely needs business data it declares a **port** in `Tf1Cbs.Abstractions` and a core module implements it — `IBroadcastSource`, `IBankVariableSource`, `IMenuSource`, `ISessionValidator`. The framework `TryAdd` s no-op defaults for the first three and resolves `ISessionValidator` optionally, so a host runs (degraded) with neither General nor Authentication present.

- **internal types** (invisible to screens): `CbsAccessMiddleware`, `UrlNormalizer`.
- The rest of the framework surface is **public by necessity** — screens genuinely inject and use it

(search & session models, the picker models, `EditViewModel`, `Notify`, `CbsSessionStore`, `RowToken`, `SearchEndpoint`, `MenuService`, `DatabaseSelector`, `CbsAccessAttributes`, `CbsScreenController`). That public set **is** the curated API; what is hidden is the plumbing and the one-call wiring.

- **XML documentation is generated** for the framework (and the eight other assemblies that ship as prebuilt DLLs to an outside team), so hovering a type in the editor shows its semantics even with no source present.

## Enforcement in `Tf1Cbs.ArchTests`

---

The cross-assembly boundaries are compiler-guaranteed, so the arch suite's job is to notice when a guarantee **changes level** rather than to be the guarantee itself:

- `Framework_must_not_reference_any_business_module_assembly` asserts on

`GetReferencedAssemblies()` — re-adding a reference fails loudly and says why.

- `Framework_must_consume_business_modules_only_through_abstraction_ports` is a denylist of named concrete types, backed by the assembly-level rule above.

- `BoundaryGuaranteeTests` re-verifies every boundary's declared enforcement level

(`Enforcement.Namespace` vs `Enforcement.Reference`) against the live reference graph, so a namespace rule cannot quietly become a tautology when a project reference disappears. See [modular-monolith.md](#).

- `NamespaceDisciplineTests` enforces `namespace == assembly`.

## Resolved: the shared-view RCL

---

This document previously deferred `Tf1Cbs.Web.Shared` on the grounds that `internal` hiding does not apply to `.cshtml`, so an RCL would buy *sharing*, not *hiding* — and there was only one host. **Four thin hosts and a gateway later, sharing is the whole point**, and the RCL was extracted. The three gotchas anticipated then all applied and were handled:

1. RCL `wwwroot` serves from `~/_content/Tf1Cbs.Web.Shared/...; _Layout / _LayoutBlank` asset

references were updated accordingly (and each screen RCL serves its own JS from `~/_content/Tf1Cbs.Modules.<Area>.Web/`).

1. The shared `_ViewImports` `@using` set moved into an RCL-level `_ViewImports`.
2. String-named partials (`_SearchModal`, `_RecordPicker`, `_Notification`, ...) resolve from the RCL

automatically as compiled Razor views — no per-call changes.

## Extending the framework

---

Reusable components stay **additive with safe defaults**: a new property on `RecordPickerModel`, `SearchDescriptor` or `DatePickerModel` must leave every existing call site rendering exactly as before. When a shared component is added or extended, run the **component-docs-sync** agent (keeps `docs/legacy-vs-new-components.*` honest) and the **component-demo** agent (keeps the `/Dev/Components` showcase covering the full surface).